Практичне заняття № 1 на тему «Попередня обробка даних. Профайлінг даних. Обробка некоректних даних».

Навчальна мета заняття: формування практичних навичок попередньої обробка даних: профайлінгу даних і обробки пропусків даних.

Кількість годин – 4 години.

Місце проведення – навчальна аудиторія або комп'ютерний клас.

Навчальні питання:

1. Профайлінг даних (Data Profiling) – загальні відомості.
2. Основні етапи data profiling.
3. Методи профілювання даних.
4. Інструменти для data profiling.
5. Попередня обробка даних (Препроцесінг). Етапи.
6. Обробка дублікатів, некоректних даних.
7. Обробка пропусків даних.

Література.

1. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студ. спеціальності 122 "Комп'ютерні науки", спеціалізації "Інформаційні технології в біології та медицині" / А. В. Яковенко ; КПІ ім. Ігоря Сікорського. – Електронні текстові данні (1 файл: 1,59 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2018. – 195 с.

2. Болюбаш Н. М. Інтелектуальний аналіз даних : навч. посіб. / Н. М. Болюбаш. - Миколаїв : Вид-во ЧНУ ім. Петра Могили, 2023. - 320 с.

3. Кононова К. Ю. Машинне навчання: методи та моделі: підручник для бакалаврів, магістрів та докторів філософії спеціальності 051 «Економіка» / К. Ю. Кононова. – Харків: ХНУ імені В. Н. Каразіна, 2020. – 301 с.

4. Charu C. Aggarwal. Data Mining: The Textbook – Cham: Springer Ltd. Publ. Switzerland, 2015. – 734p. URL: https://charuaggarwal.net/Data-Mining.pdf.

1. ПІДГОТОВКА ДО ЗАНЯТТЯ.

1.1. Вивчити сутність і етапи профайлінгу даних.
1.2. Вивчити сутність і етапи препроцесінгу даних.
1.3. Вивчити засоби і інструменти профайлінгу даних.
1.4. Вивчити методи, засоби і інструменти обробки пропусків даних.
1.5. Виконати домашнє завдання і написати звіт.

2. ПЛАН ПРОВЕДЕННЯ ЗАНЯТТЯ.

2.1. Проведення експрес-контролю готовності до заняття:

2.1.1. Перевірка виконання домашнього завдання.

2.1.2. Контрольна робота:

1 варіант: сутність і етапи профайлінгу даних;

2 варіант: методи, засоби і інструменти обробки пропусків даних.

2.1.2. Контроль знань теоретичного матеріалу.

2.2. Рішення задач профайлінгу даних:

2.2.1. Завантажити до свого середовища навчальний файл з датасетом – data1.csv.

2.2.2. Відпрацювати прийоми профайлінгу даних:

1. Перегляд даних.

Одним із найбільш використовуваних методів отримання швидкого попереднього огляду DataFrame є метод head().

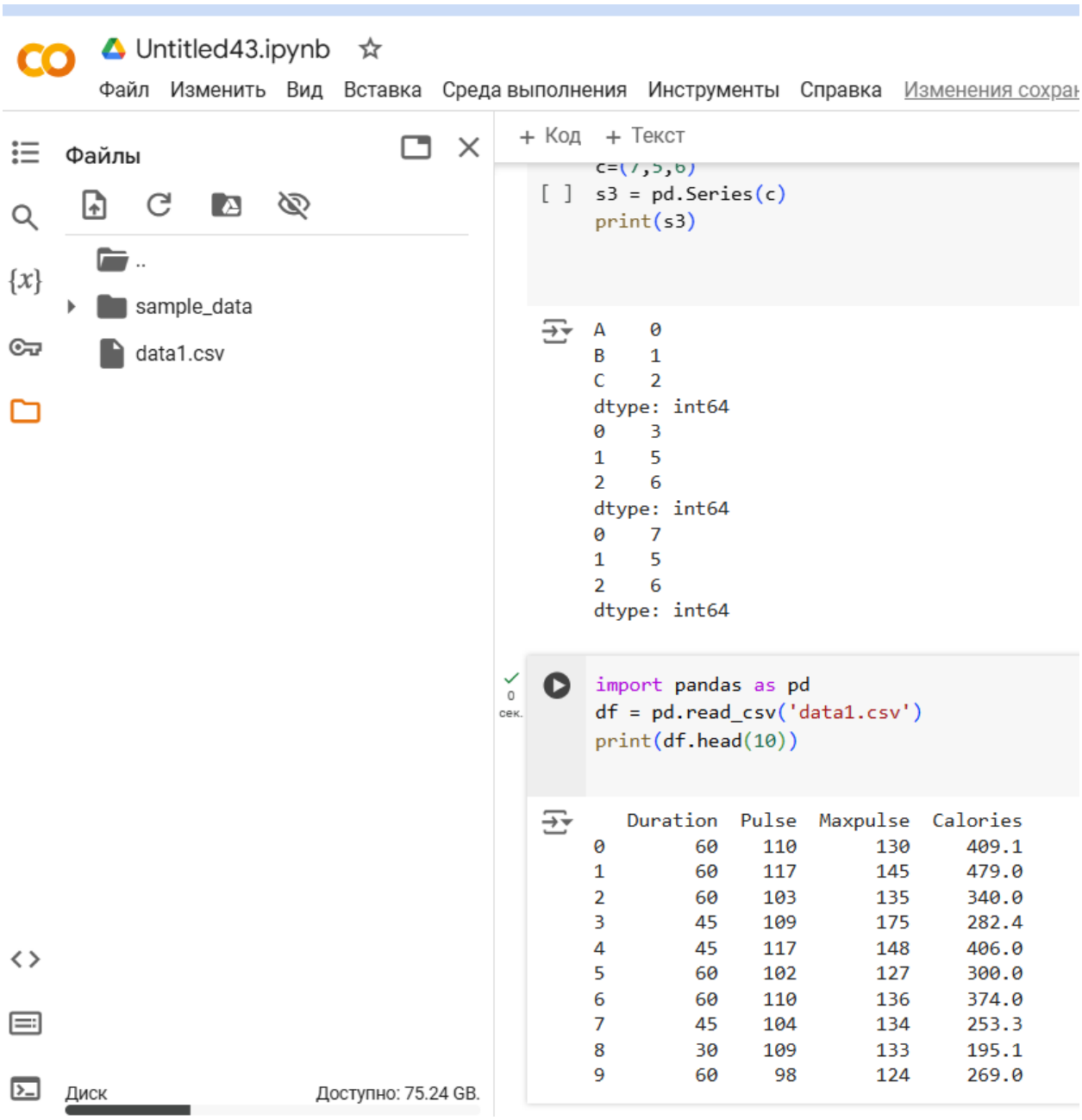
Метод head() повертає заголовки та вказану кількість рядків, починаючи з верхнього.

Приклад 1. Завантажимо навчальний датасет data1.csv у датафрейм і виконаємо попередній короткий огляд, надрукувавши перші 10 рядків DataFrame:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head(10))
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
5        60    102       127     300.5
6        60    110       136     374.0
7        45    104       134     253.3
8        30    109       133     195.1
9        60     98       124     269.0
```

Примітка: якщо кількість рядків не вказано, метод head() поверне 5 верхніх рядків - `print(df.head())`.



Надрукувати останні 10 рядків датасету:

```
df = pd.read_csv('data1.csv')
print(df.head(10))
print("-------------")
print(df.tail(10))
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
5        60    102       127     300.0
6        60    110       136     374.0
7        45    104       134     253.3
8        30    109       133     195.1
9        60     98       124     269.0
-------------
```

```
     Duration  Pulse  Maxpulse  Calories
159        30     80       120     240.9
160        30     85       120     250.4
161        45     90       130     260.4
162        45     95       130     270.0
163        45    100       140     280.9
164        60    105       140     290.8
165        60    110       145     300.0
166        60    115       145     310.2
167        75    120       150     320.4
168        75    125       150     330.4
```

Сформувати більш компактний датасет для подальшої роботи і записати його на диск:

```
df1 = df.loc[0:30]
print("-------------")
print(df1)
df1.to_csv('data31.csv', index=False)
```

2. Отримання інформації про датасет.

Об'єкт DataFrames має метод під назвою info(), який надає вам більше інформації про набір даних.

Приклад 2.

```
print(df.info())
```

Результат:

```
<class 'pandas.core.frame.DataFrame'>
  RangeIndex: 169 entries, 0 to 168
  Data columns (total 4 columns):
   #   Column    Non-Null Count  Dtype
  ---  ------    --------------  -----
   0   Duration  169 non-null    int64
   1   Pulse     169 non-null    int64
   2   Maxpulse  169 non-null    int64
   3   Calories  164 non-null    float64
  dtypes: float64(1), int64(3)
  memory usage: 5.4 KB
  None
```

Результат говорить нам, що є 169 рядків і 4 стовпці:

```
  RangeIndex: 169 entries, 0 to 168
  Data columns (total 4 columns):
```

Стовпці даних (всього 4 стовпці):

І назва кожного стовпця з типом даних:

```
#   Column    Non-Null Count  Dtype
  ---  ------    --------------  -----
```

```
 0   Duration   169 non-null    int64
 1   Pulse      169 non-null    int64
 2   Maxpulse   169 non-null    int64
 3   Calories   164 non-null    float64
```

2.3. Рішення задач обробки пропусків даних:

2.3.1. Завантажити до свого середовища навчальний файл з навчальним датасетом – data31.csv.

2.3.2. Відпрацювати прийоми обробки пропущених даних:

Метод info()повідомляє нам, скільки ненульових значень міститься в кожному стовпці, і в нашому наборі даних здається, що в стовпці «Калорії» є 164 із 169 ненульових значень. Це означає, що в стовпці «Калорії» з будь-якої причини є 5 рядків без жодного значення (пропуски).

1. Видалення рядків, що містять пропуски.

Один із найпростіших способів роботи з порожніми клітинками — видалити рядки, які містять порожні клітинки.

Зазвичай це нормально, оскільки набори даних можуть бути дуже великими, і видалення кількох рядків не матиме великого впливу на результат.

В наступних прикладах знов будемо використовувати наш навчальний датасет data31.csv:

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	2020/12/26	100	120	250.0

```
  27         60  '2020/12/27'     92        118      241.0
  28         60  '2020/12/28'    103        132        NaN
  29         60  '2020/12/29'    100        132      280.0
  30         60  '2020/12/30'    102        129      380.3
  31         60  '2020/12/31'     92        115      243.0
```

Застосуємо до нього метод .info():

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31 entries, 0 to 30
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  31 non-null     int64
 1   Pulse     31 non-null     int64
 2   Maxpulse  31 non-null     int64
 3   Calories  29 non-null     float64
dtypes: float64(1), int64(3)
     memory usage: 1.1 KB
```

Приклад 1. Повернути новий фрейм даних без порожніх комірок:

```
import pandas as pd
df = pd.read_csv('data.csv')
print()
print(new_df.to_string())
new_df = df.dropna()
print(new_df.to_string())
```

```
Duration          Date  Pulse  Maxpulse  Calories
0          60  '2020/12/01'    110       130     409.1
1          60  '2020/12/02'    117       145     479.0
2          60  '2020/12/03'    103       135     340.0
3          45  '2020/12/04'    109       175     282.4
4          45  '2020/12/05'    117       148     406.0
5          60  '2020/12/06'    102       127     300.0
6          60  '2020/12/07'    110       136     374.0
7         450  '2020/12/08'    104       134     253.3
8          30  '2020/12/09'    109       133     195.1
9          60  '2020/12/10'     98       124     269.0
10         60  '2020/12/11'    103       147     329.3
11         60  '2020/12/12'    100       120     250.7
12         60  '2020/12/12'    100       120     250.7
13         60  '2020/12/13'    106       128     345.3
14         60  '2020/12/14'    104       132     379.3
15         60  '2020/12/15'     98       123     275.0
16         60  '2020/12/16'     98       120     215.2
17         60  '2020/12/17'    100       120     300.0
19         60  '2020/12/19'    103       123     323.0
20         45  '2020/12/20'     97       125     243.0
21         60  '2020/12/21'    108       131     364.2
23         60  '2020/12/23'    130       101     300.0
24         45  '2020/12/24'    105       132     246.0
25         60  '2020/12/25'    102       126     334.5
26         60    2020/12/26    100       120     250.0
27         60  '2020/12/27'     92       118     241.0
29         60  '2020/12/29'    100       132     280.0
```

```
30          60  '2020/12/30'    102        129      380.3
31          60  '2020/12/31'     92        115      243.0
```

Зверніть увагу на результат, що деякі рядки було видалено (рядки 18, 22 і 28). Ці рядки містили клітинки з порожніми значеннями.

Примітка. За замовчуванням ***метод dropna() повертає новий DataFrame*** **і не змінює оригінальний**.

Якщо ви хочете змінити вихідний DataFrame, а не створювати новий, використовуйте аргумент inplace = True.

Приклад. Видалити всі рядки зі значеннями NaN:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace = True)
print(df.to_string())
```

2. Заміна порожніх значень.

Інший спосіб роботи з порожніми клітинками — вставити замість них нове значення. Таким чином вам не доведеться видаляти цілі рядки лише через деякі порожні клітинки.

Метод fillna() дозволяє нам замінити порожні клітинки значенням, наприклад: замініть значення NULL на число 130:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna(130, inplace = True)
```

```
Duration          Date  Pulse  Maxpulse  Calories
0          60  '2020/12/01'    110       130      409.1
1          60  '2020/12/02'    117       145      479.0
2          60  '2020/12/03'    103       135      340.0
3          45  '2020/12/04'    109       175      282.4
4          45  '2020/12/05'    117       148      406.0
5          60  '2020/12/06'    102       127      300.0
6          60  '2020/12/07'    110       136      374.0
7         450  '2020/12/08'    104       134      253.3
8          30  '2020/12/09'    109       133      195.1
9          60  '2020/12/10'     98       124      269.0
10         60  '2020/12/11'    103       147      329.3
11         60  '2020/12/12'    100       120      250.7
12         60  '2020/12/12'    100       120      250.7
13         60  '2020/12/13'    106       128      345.3
14         60  '2020/12/14'    104       132      379.3
15         60  '2020/12/15'     98       123      275.0
16         60  '2020/12/16'     98       120      215.2
17         60  '2020/12/17'    100       120      300.0
18         45  '2020/12/18'     90       112      130.0
19         60  '2020/12/19'    103       123      323.0
20         45  '2020/12/20'     97       125      243.0
21         60  '2020/12/21'    108       131      364.2
22         45           130    100       119      282.0
```

```
23          60   '2020/12/23'    130        101      300.0
24          45   '2020/12/24'    105        132      246.0
25          60   '2020/12/25'    102        126      334.5
26          60     2020/12/26    100        120      250.0
27          60   '2020/12/27'     92        118      241.0
28          60   '2020/12/28'    103        132      130.0
29          60   '2020/12/29'    100        132      280.0
30          60   '2020/12/30'    102        129      380.3
31          60   '2020/12/31'     92        115      243.0
```

Замінити лише для вказаних стовпців

Наведений вище приклад замінює всі порожні клітинки в усьому датафреймі.

Щоб замінити порожні значення лише для одного стовпця, необхідно вказати назву стовпця для DataFrame:

Замініть значення NULL у стовпцях «Калорії» на число 130:

```
import pandas as pd
df = pd.read_csv('data.csv')
df["Calories"].fillna(130, inplace = True)
```

3. Заміна за допомогою середнього значення, медіани або моди.

Поширеним способом заміни порожніх клітинок є обчислення середнього, медіани або модового значення стовпця.

Pandas використовує методи mean(), median() і mode() для обчислення відповідних значень для вказаного стовпця.

Приклад.

Обчисліть середнє значення та замініть ним будь-які порожні значення:

```
df = pd.read_csv('data.csv')
x_mean = df["Calories"].mean()
print("mean = ", x)
df["Calories"].fillna(x_mean, inplace = True)
print(df.to_string())
mean =  304.68
Duration          Date  Pulse  Maxpulse  Calories
0          60   '2020/12/01'    110        130     409.10
1          60   '2020/12/02'    117        145     479.00
2          60   '2020/12/03'    103        135     340.00
3          45   '2020/12/04'    109        175     282.40
4          45   '2020/12/05'    117        148     406.00
5          60   '2020/12/06'    102        127     300.00
6          60   '2020/12/07'    110        136     374.00
7         450   '2020/12/08'    104        134     253.30
8          30   '2020/12/09'    109        133     195.10
9          60   '2020/12/10'     98        124     269.00
10         60   '2020/12/11'    103        147     329.30
11         60   '2020/12/12'    100        120     250.70
```

```
12         60  '2020/12/12'    100        120    250.70
13         60  '2020/12/13'    106        128    345.30
14         60  '2020/12/14'    104        132    379.30
15         60  '2020/12/15'     98        123    275.00
16         60  '2020/12/16'     98        120    215.20
17         60  '2020/12/17'    100        120    300.00
18         45  '2020/12/18'     90        112    304.68
19         60  '2020/12/19'    103        123    323.00
20         45  '2020/12/20'     97        125    243.00
21         60  '2020/12/21'    108        131    364.20
22         45           NaN    100        119    282.00
23         60  '2020/12/23'    130        101    300.00
24         45  '2020/12/24'    105        132    246.00
25         60  '2020/12/25'    102        126    334.50
26         60    2020/12/26    100        120    250.00
27         60  '2020/12/27'     92        118    241.00
28         60  '2020/12/28'    103        132    304.68
29         60  '2020/12/29'    100        132    280.00
30         60  '2020/12/30'    102        129    380.30
31         60  '2020/12/31'     92        115    243.00
```

Як ви можете бачити в рядках 18 і 28, порожні значення з «Калорій» було замінено на середнє: 304,68.

Приклад.

Обчисліть медіану і замініть нею будь-які порожні значення (медіана = значення посередині після того, як ви відсортували всі значення за зростанням):

```
import pandas as pd
df = pd.read_csv('data.csv')
x_median = df["Calories"].median()
print("median = ", x_median)
df["Calories"].fillna(x_median, inplace = True)
print(df.to_string())
```

```
median =  291.2
    Duration          Date  Pulse  Maxpulse  Calories
0         60  '2020/12/01'    110       130     409.1
1         60  '2020/12/02'    117       145     479.0
2         60  '2020/12/03'    103       135     340.0
3         45  '2020/12/04'    109       175     282.4
4         45  '2020/12/05'    117       148     406.0
5         60  '2020/12/06'    102       127     300.0
6         60  '2020/12/07'    110       136     374.0
7        450  '2020/12/08'    104       134     253.3
8         30  '2020/12/09'    109       133     195.1
9         60  '2020/12/10'     98       124     269.0
10        60  '2020/12/11'    103       147     329.3
11        60  '2020/12/12'    100       120     250.7
12        60  '2020/12/12'    100       120     250.7
13        60  '2020/12/13'    106       128     345.3
14        60  '2020/12/14'    104       132     379.3
```

```
15        60  '2020/12/15'     98       123     275.0
16        60  '2020/12/16'     98       120     215.2
17        60  '2020/12/17'    100       120     300.0
18        45  '2020/12/18'     90       112     291.2
19        60  '2020/12/19'    103       123     323.0
20        45  '2020/12/20'     97       125     243.0
21        60  '2020/12/21'    108       131     364.2
22        45           NaN    100       119     282.0
23        60  '2020/12/23'    130       101     300.0
24        45  '2020/12/24'    105       132     246.0
25        60  '2020/12/25'    102       126     334.5
26        60      20201226    100       120     250.0
27        60  '2020/12/27'     92       118     241.0
28        60  '2020/12/28'    103       132     291.2
29        60  '2020/12/29'    100       132     280.0
30        60  '2020/12/30'    102       129     380.3
31        60  '2020/12/31'     92       115     243.0
```

Як ви можете бачити в рядках 18 і 28, порожні значення з «Калорій» було замінено медіаною: 291,2

Заміна порожніх значень модою.

Приклад.

Обчисліть моду і замініть ним будь-які порожні значення (мода = значення, яке з'являється найчастіше):

```
df = pd.read_csv('data31.csv')
x_mode = df["Calories"].mode()[0]
print("mode = ", x_mode)
#df["Calories"].fillna(x_mode, inplace = True) # - застаріла
         нотація
df.fillna({"Calories":x_mode}, inplace = True)
print(df.to_string())
```

Домашнє завдання 1:

Завантажити з Kaggle який-небудь датасет (наприклад, «Титанік»). Опрацювати його за наведеною вище методикою.

Можете взяти датасет Іріси Фішера на г-диску або інший, наприклад, популярний датасет Penguins (Пінгвіни) бібліотеки Seaborn.